PROJECT REPORT

GEOSPATIAL QUERY SYSTEM

A Flask-Based Natural Language Location Search Application

Submitted in partial fulfillment of the requirements for the course project

Academic Year: 2025–26

Masters in Computer Applications

Rungta International Skills University

Contents

Abstract

The Geospatial Query System is a Flask-based web application that enables users to search for nearby real-world places using natural language queries such as “Find hospitals in Delhi” or “Find cafes nearby.” The project integrates natural language processing, geocoding, OpenStreetMap data, and interactive map visualization to deliver a simple and user-friendly location search experience.

The system extracts the required place type and location from user queries using spaCy, converts city names into geographic coordinates through Nominatim, retrieves relevant places through geopy, and presents the results as responsive cards containing names, addresses, latitude, and longitude. It also supports current-location search through browser GPS and generates an interactive Folium map with markers for all discovered places.

This project is useful because location search is a common daily requirement, and a natural language interface reduces manual steps while making the process faster, easier, and more accessible.

1. Introduction

Location-based applications are widely used for finding hospitals, restaurants, parks, pharmacies, transport stations, and other nearby facilities. Traditional search interfaces often require users to type exact keywords, manually apply filters, or know the precise spelling of a place name. This project offers a more intuitive interface where the user can enter a natural language sentence and the system automatically identifies the relevant search details.

The application is implemented as a modular Python Flask project with dedicated components for NLP processing, geolocation, query execution, and map generation. The frontend uses HTML, CSS, Bootstrap, and JavaScript to deliver a responsive search interface with GPS support, animated placeholders, result cards, and an embedded interactive map.

1.1 Objectives

- Extract place type and location from natural language user input.
- Search for places using OpenStreetMap/Nominatim data.
- Support both city-based search and browser current-location search.
- Display useful details including name, full address, latitude, and longitude.
- Generate an interactive map with clickable location markers.
- Reduce manual effort in searching for nearby facilities.

2. Scope

The project is suitable for students, travellers, and general users who need a quick way to locate common facilities in Indian cities or near their current location. It supports searches across a wide range of categories, including hospitals, schools, restaurants, cafes, banks, pharmacies, parks, libraries, hotels, supermarkets, airports, bus stations, railway stations, metro stations, cinemas, tourist attractions, and clinics.

The current version is a functional web prototype dependent on Nominatim/geopy API responses. Future iterations can incorporate database caching, advanced result ranking, distance filtering, and production-ready mapping APIs.

3. System Requirements

Table 1: System Requirements

Category	Requirement
Hardware	Computer or laptop with minimum 4 GB RAM and a stable internet connection
Operating System	Windows, Linux, or macOS
Programming Language	Python 3.x
Framework	Flask
Libraries	spaCy, geopy, folium, requests, and supporting Python packages
Frontend	HTML, CSS, JavaScript, and Bootstrap
Data Source	OpenStreetMap through the Nominatim geocoder
Browser Permission	Location permission for current-location search

4. Methodology

1. The user opens the Flask web application and enters a query in natural language.
2. JavaScript captures GPS latitude and longitude if the user selects current-location search.
3. The Flask route receives the form data and passes the query to the NLP processor.
4. The NLP processor cleans the query, detects location entities, and maps keywords to supported place types.
5. For city-based search, the geolocator module converts the location name into coordinates.
6. The query engine searches for relevant places through Nominatim and collects name, address, and coordinate details.
7. The map generator creates a Folium map centered on the first result and adds markers for all returned places.
8. The frontend displays extracted NLP data, coordinates, result cards, and the interactive map.

5. Process Flowchart

Start → User Query / Current Location → Flask Request → NLP Extraction → Geocoding → Place Search → Generate Map → Display Results → **End**

Table 2: Process Steps

Step	Process
1	Start application
2	User enters query or selects current location
3	Flask receives the HTTP request
4	NLP module extracts place type and location
5	System checks whether the location is current location or a city name
6	Coordinates are obtained using browser GPS or Nominatim geocoding
7	Nearby places are searched
8	Map markers and result cards are generated
9	Results are displayed to the user
10	End

6. Module and Class Diagram

Note: This project is modular and function-driven rather than class-based. Therefore, the following table presents module responsibilities and key functions instead of a traditional UML class diagram.

Table 3: Module Responsibilities

Module / File	**Responsibility**	**Key Functions / Elements**
`app.py`	Flask routing and request handling	`home()`, `render_template()`, request processing
`modules/ nlp_processor.py`	Extracts place type and location from user query	`extract_query_data()`, `PLACE_TYPES`
`modules/ geolocator.py`	Converts location names into coordinates	`get_coordinates()`
`modules/ query_engine.py`	Searches nearby places through Nominatim/geopy	`search_places()`, `search_places_nearby()`, `SEARCH_MAPPINGS`
`modules/ map_generator.py`	Creates an interactive map and markers	`generate_map()`
`templates/ index.html`	Displays UI, search form, result cards, and map iframe	Jinja conditions, result loop, form fields
`static/js/ location.js`	Handles browser geolocation	`getLocation()`

6.1 Project Structure

The project follows a modular directory layout:

```
geospatial_project/
|-- app.py
|-- modules/
|   |-- nlp_processor.py
|   |-- geolocator.py
|   |-- query_engine.py
|   |-- map_generator.py
|-- templates/
|   |-- index.html
|-- static/
|   |-- css/style.css
|   |-- js/location.js
|   |-- images/
|-- screenshots/
|-- requirements.txt
```

7. Implementation

The implementation is divided into independent modules, each with a clearly defined responsibility. The main Flask application handles GET and POST requests on the home route. On a POST request, it reads the submitted query along with optional latitude and longitude values, then delegates to the NLP module to identify the location and requested place category.

The NLP processor uses spaCy for named entity recognition and a dictionary of supported place types. If the query contains the word “nearby,” the location is treated as the user’s current position. Otherwise, the detected city or place name is forwarded to the geolocation module.

The geolocator module uses Nominatim to convert a location name into latitude and longitude coordinates. The query engine then constructs a search phrase, applies smart mappings such as train station to railway station and movie theater to cinema, and returns up to ten matching places with their names, addresses, and coordinates.

The map generator uses Folium to produce an HTML map file saved at `static/maps/map.html`. Each result is represented as a marker with a popup and tooltip. The frontend renders this map through an iframe and displays every place in a responsive card layout.

8. Sample Input / Output

Table 4: Sample Input and Expected Output

Input Query	**Expected Output**
Find hospitals in Delhi	Hospital listings in Delhi with address, coordinates, and map markers
Find cafes nearby	Cafe results based on current browser GPS location
Restaurants in Mumbai	Restaurant listings in Mumbai with full details
Find parks in Jaipur	Park results around Jaipur with interactive map

9. Project Screenshots

The following screenshots illustrate the application interface and output at various stages.

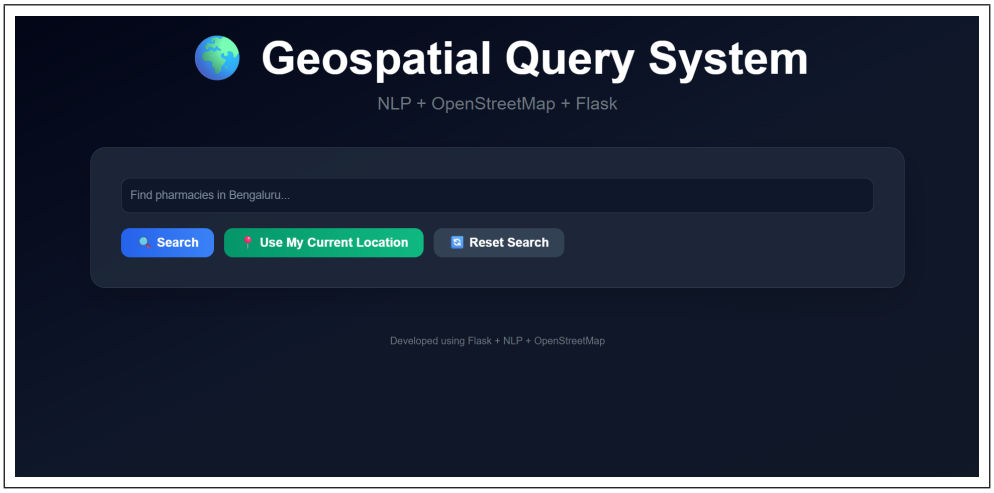

Figure 1: Homepage UI - main search interface with animated placeholder and GPS option.

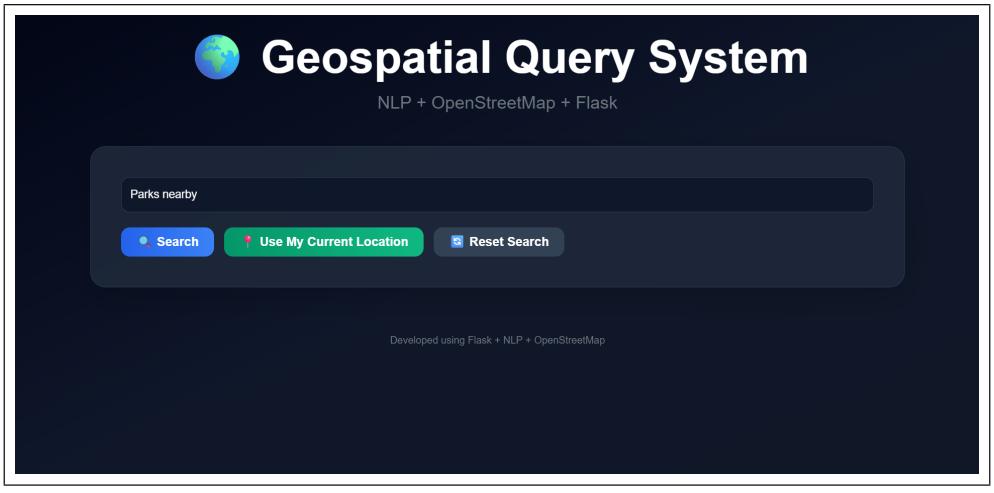

Figure 2: Search Query Demo - a sample query being entered and processed.

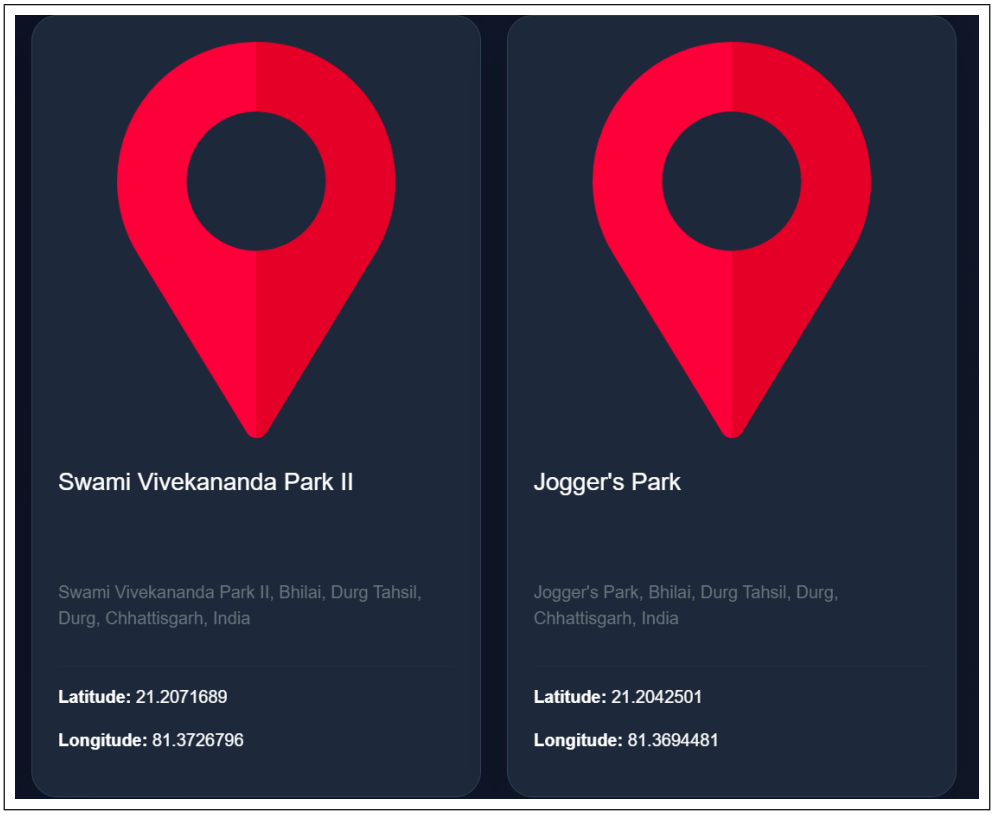

Figure 3: Nearby Places Result Cards - structured result cards showing name, address, and coordinates.

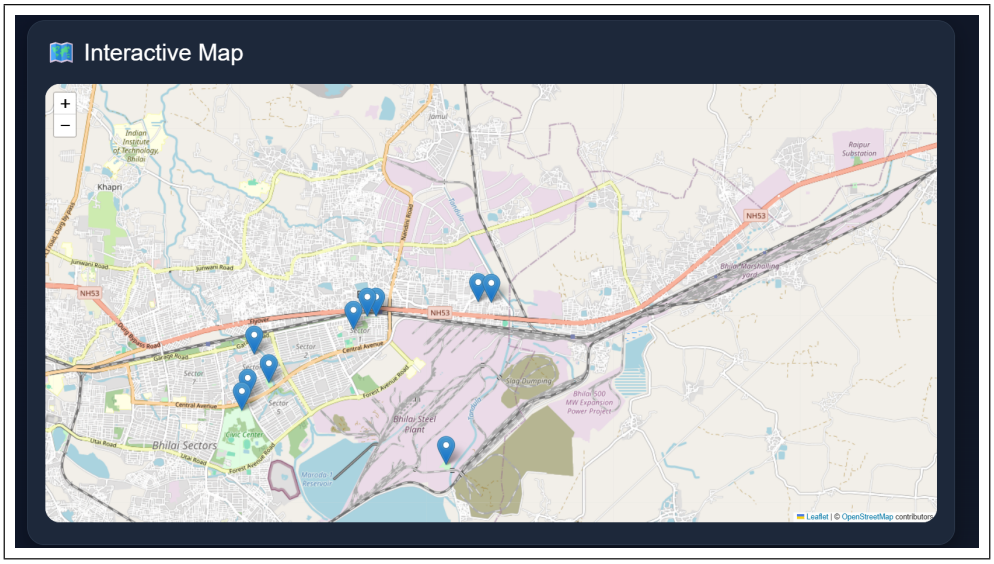

Figure 4: Interactive Map with Markers - Folium map displaying discovered locations.

10. Future Enhancements

- Add voice-based location search for hands-free interaction.
- Implement distance filtering and nearest-result sorting.
- Cache geocoding results to improve speed and reduce redundant API calls.
- Incorporate user ratings, opening hours, and category icons where available.
- Expand query understanding with additional place categories and synonym support.
- Add user authentication and a saved favorite places feature.
- Convert the project into a mobile application or Progressive Web App.
- Integrate Google Maps or another production-grade map provider for higher reliability.

11. Project Timeline (Gantt Chart)

Table 5: Project Timeline

Task	**Week 1**	**Week 2**	**Week 3**	**Week 4**
Requirement analysis and topic selection	Yes			
UI design and Flask setup	Yes	Yes		
NLP extraction and geolocation modules		Yes	Yes	
Map generation and result display			Yes	
Testing, screenshots, and documentation				Yes

12. Conclusion

The Geospatial Query System successfully demonstrates how natural language processing and geospatial services can be combined in a practical, real-world web application. By accepting simple user queries, extracting search intent, discovering relevant places, and presenting results through both structured cards and an interactive map, the system delivers a useful location search experience.

The project showcases the integrated use of Python, Flask, spaCy, geopy, Nominatim, Folium, and modern frontend technologies within a single cohesive system. It also establishes a solid foundation for future enhancements such as improved result ranking, distance-based filtering, voice search, and mobile application support.

References

[1] Flask Documentation. [Online]. Available: https://flask.palletsprojects.com/

[2] spaCy Documentation. [Online]. Available: https://spacy.io/usage

[3] geopy Documentation. [Online]. Available: https://geopy.readthedocs.io/

[4] Folium Documentation. [Online]. Available: https://python-visualization.github.io/folium/

[5] OpenStreetMap. [Online]. Available: https://www.openstreetmap.org/

[6] Nominatim Documentation. [Online]. Available: https://nominatim.org/release-docs/latest/